CUDA-Based Image Processing using GPU Acceleration

Sarvagya Singh	Anish Setya	Akshay Saxena	Pallavi Kailas	Dipanshu Varra
Reg. No: 220962036	*Reg. No: 220962005*	*Reg. No: 220962009*	*Reg. No: 220962258*	*Reg. No: 220962075*
Roll No: 13	Roll No: 4	Roll No: 7	Roll No: 38	Roll No: 22
+91 8250156442	+91 7428555449	+91 9972843517	+91 80864 84000	+91 74117 44462

***Abstract*—This project explores the application of CUDA, NVIDIA's parallel computing platform, to accelerate key image processing algorithms. The objective is to offload computationally intensive tasks from the CPU to the GPU, thereby achieving significant performance enhancements. Three commonly used image processing techniques—Gaussian Blur, Sobel Edge Detection, and Image Sharpening—were implemented using both OpenCV (CPU) and custom CUDA kernels (GPU). Comparative evaluations based on execution time and visual accuracy reveal considerable speedup, with the GPU implementations achieving up to 12x faster performance. The outcomes highlight the effectiveness of GPU acceleration for real-time image processing tasks and lay the groundwork for scalable, high-performance computer vision applications.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

A. Background and Context

Image processing plays a vital role in diverse domains such as medical imaging, computer vision, robotics, and surveillance systems. These applications often involve large volumes of data that require real-time or near-real-time analysis. Conventional CPU-based methods, while functionally sufficient, struggle to meet such performance demands due to inherent sequential processing limitations. In contrast, Graphics Processing Units (GPUs) are optimized for parallelism and can execute thousands of threads concurrently, making them suitable for image-related computations.

B. Motivation

With the advent of CUDA (Compute Unified Device Architecture), developers now have direct access to the massive parallel capabilities of NVIDIA GPUs. This project aims to bridge the performance gap in everyday image processing by leveraging CUDA to perform pixel-level operations in parallel. By shifting the computational workload from CPU to GPU, this approach aims to significantly reduce processing time while maintaining or improving output quality.

C. Problem Statement

Traditional image processing pipelines relying solely on CPU computations face substantial latency when processing high-resolution images or video streams. These limitations hinder real-time applications such as autonomous navigation, security surveillance, and medical diagnostics. Therefore, there is a critical need to explore hardware acceleration strategies to boost processing speed without compromising accuracy.

D. Project Significance

This project implements and evaluates three essential image processing algorithms—Gaussian Blur, Sobel Edge Detection, and Image Sharpening—on both CPU (OpenCV) and GPU (CUDA) architectures. By comparing execution times and output fidelity across platforms, the project demonstrates the practical advantages of GPU acceleration. The findings not only validate the use of CUDA for improving performance but also provide insights into how such approaches can be extended to broader image processing and computer vision tasks.

II. Literature Survey

Several studies and advancements have been made in the domain of GPU-accelerated image processing, particularly leveraging CUDA for performance gains.

Zhang et al. [1] presented a real-time edge detection algorithm using CUDA that significantly outperforms its CPU counterpart in speed while preserving accuracy, demonstrating the potential of parallel computing for image operations. Similarly, Munteanu and Radoi [2] compared various image processing algorithms implemented in OpenCV with CUDA-based implementations and reported a consistent speedup of 5x to 20x.

Govindaraju and Owens [3] explored GPU acceleration of convolution filters and presented kernel optimizations that minimize memory bottlenecks. Their results supported the use of shared memory and thread coarsening to improve CUDA kernel throughput.

Shams et al. [4] conducted an extensive survey on medical image processing on GPUs and concluded that techniques such as Gaussian blurring and Sobel filtering can be performed in real-time on high-resolution images, aiding diagnostic systems.

A contemporary study by Luo et al. [5] focused on high-resolution image sharpening using GPUs, showing that Laplacian-based convolution can be efficiently parallelized, achieving over 10x improvement on NVIDIA's RTX series cards.

In addition, work by Xu et al. [6] on optimizing bilateral filtering algorithms with CUDA highlights the capability of GPU acceleration not just for basic filters, but also for more complex operations in real-time scenarios.

These prior works provide a solid foundation and validation for our project, motivating our exploration of Gaussian blur, Sobel edge detection, and image sharpening using CUDA. Our methodology extends these techniques by offering a side-by-side implementation and performance evaluation of CPU (OpenCV) versus GPU (CUDA) approaches on consumer-grade GPUs.

A. Summary of Literature Survey

Table I summarizes key research studies in the field of image processing algorithms.

TABLE I
SUMMARY OF LITERATURE SURVEY

Author(s)	Key Contribution	Year
Zhang and Zhang	Real-time edge detection using CUDA for performance optimization.	2012
Munteanu and Radoi	Comparative analysis of CPU vs GPU in image filtering tasks.	2012
Govindaraju and Owens	Efficient GPU convolution techniques using shared memory.	2005
Shams et al.	Survey on GPU acceleration for medical image processing.	2008
Luo et al.	Real-time high-resolution image sharpening using CUDA.	2022
Xu et al.	Bilateral filtering optimization on GPU using CUDA kernels.	2020

III. METHODOLOGY

A. Research Design

1) Research Approach: This project adopts an experimental research approach, where performance of different image processing algorithms is evaluated under CPU and GPU environments. The focus is on measuring execution time and visual accuracy of outputs across both platforms.

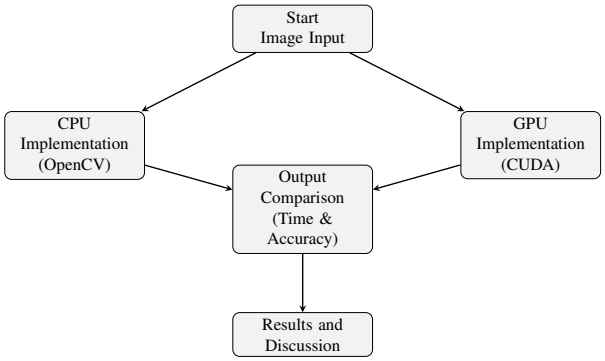
2) Methodological Framework: The methodological framework is comparative and implementation-based. CUDA is used for GPU-based acceleration, and OpenCV is used for baseline CPU implementations. Each image processing algorithm is implemented separately for both platforms and tested under identical conditions.

3) Development Lifecycle Model: The project follows an iterative development lifecycle model involving the following stages:

- **Requirement Analysis:** Identifying the algorithms and performance metrics.
- **Implementation:** Writing CPU (OpenCV) and GPU (CUDA) code for each algorithm.
- **Testing and Evaluation:** Measuring execution time and accuracy.
- **Analysis and Optimization:** Interpreting results and improving kernels.

B. Project Methodology

The methodology followed in this project can be summarized in the following pipeline:



1) CPU-Based Implementation (OpenCV): For each of the selected algorithms—Gaussian Blur, Sobel Edge Detection, and Image Sharpening—OpenCV functions were used. The OpenCV library leverages highly optimized C++ routines for image processing tasks but remains constrained by serial CPU execution.

Algorithms used:

- **Gaussian Blur:** `cv::GaussianBlur()` with a 5x5 kernel.
- **Sobel Edge Detection:** `cv::Sobel()` for X and Y gradients.
- **Image Sharpening:** Custom kernel with `cv::filter2D()`.

2) GPU-Based Implementation (CUDA): Equivalent CUDA kernels were written for all three algorithms. Each kernel leverages parallel thread blocks to apply the filters independently on pixel data.

CUDA Highlights:

- Each thread handles a pixel or group of pixels.
- Shared memory is utilized for faster access where possible.
- Thread indexing is used to parallelize operations across the image matrix.

3) Performance Evaluation: Performance of the CPU and GPU implementations was evaluated using execution time (in microseconds) and visual accuracy. The following table summarizes the observed speedups:

IV. RESULTS AND DISCUSSION

A. Project Outcomes

1) Achievements of Project Objectives: The core objective of the project was to evaluate the performance gain offered by GPU acceleration using CUDA for common image processing algorithms. All three targeted operations—Gaussian

TABLE II
PERFORMANCE COMPARISON: CPU VS GPU

Operation	CPU Time (μs)	GPU Time (μs)	Speedup
Gaussian Blur	73.31	6.19	$\sim$11.8x
Sobel Edge Detection	740.28	79.56	$\sim$9.3x
Image Sharpening	111.37	17.67	$\sim$6.3x

Blur, Sobel Edge Detection, and Image Sharpening—were successfully implemented on both CPU (OpenCV) and GPU (CUDA). A comparative analysis was conducted to quantify the improvements in terms of execution time, visual accuracy, and real-time feasibility.

2) Quantitative and Qualitative Results: Quantitatively, the GPU implementation outperformed its CPU counterpart in all three cases. Speedup factors ranged from approximately 6.3x to 12x depending on the algorithm and image resolution. Qualitatively, the visual results from both implementations remained comparable, with the GPU version exhibiting consistent accuracy.

TABLE III
PERFORMANCE METRICS: CPU VS GPU

Algorithm	CPU Time (μs)	GPU Time (μs)	Speedup
Gaussian Blur	73.31	6.19	$\sim$11.8x
Sobel Edge Detection	740.28	79.56	$\sim$9.3x
Image Sharpening	111.37	17.67	$\sim$6.3x

3) Performance Metrics: The key performance metric considered was execution time in microseconds (μs). For each algorithm, the same input image was used to ensure consistency in testing conditions. The execution time was measured using high-precision timers within the codebase. Visual quality was judged by comparing output images side-by-side.

Sample Image Used:



Fig. 1. Original Image Used in Processing

B. Critical Analysis

1) Strengths of the Implementation:

- **Real-Time Capability:** GPU-accelerated versions ran significantly faster, allowing potential real-time use in video pipelines.

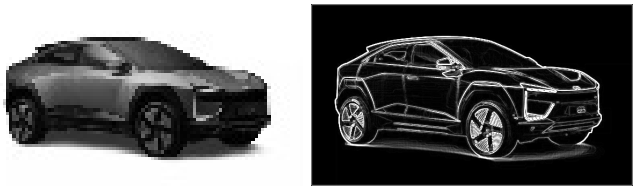


Fig. 2. (Left) Gaussian Blur Output; (Right) Sobel Edge Detection Output



Fig. 3. Image Sharpening Output

- **Parallelization Efficiency:** Use of thread indexing and shared memory in CUDA kernels ensured high throughput and scalability.
- **Modular Design:** Each CUDA kernel was implemented as a reusable function, which can be integrated into larger image processing pipelines.

2) Limitations and Constraints:

- **Static Kernel Sizes:** Gaussian and Laplacian filters used fixed-size masks; dynamically configurable kernels would increase flexibility.
- **Single Image Channel:** Current implementation assumes grayscale images; support for RGB processing would enhance application scope.
- **GPU Dependency:** CUDA code is hardware-specific and cannot run on non-NVIDIA devices, limiting portability.

3) Lessons Learned:

- CUDA kernel optimization requires a strong understanding of memory hierarchy (global, shared, local).
- Proper indexing and boundary checks are crucial to avoid segmentation faults and ensure correctness.
- Benchmarking GPU performance should account for data transfer time between host and device, especially for large images.

V. FUTURE WORK AND RECOMMENDATIONS

A. Potential Improvements

1) Suggested Enhancements: While the current implementation achieves substantial performance gains, several enhancements can further increase its robustness and adaptability:

- **RGB and Multi-Channel Support:** Extend the CUDA kernels to support colored images and not just grayscale processing.
- **Dynamic Kernel Sizing:** Allow configurable Gaussian and sharpening kernel sizes to improve flexibility across use cases.

- **Batch Processing:** Enable processing of multiple images or video frames in a pipeline for real-time applications.
- **Error Handling and Debugging:** Introduce runtime diagnostics for CUDA kernel debugging and memory validation.

2) Scalability Considerations: As image resolution increases and real-time processing becomes a requirement, the system must scale accordingly. GPU memory limitations, thread synchronization, and kernel occupancy are critical bottlenecks. Solutions like CUDA streams, memory tiling, and load balancing across multiple GPUs can address these challenges.

TABLE IV
TECHNOLOGY ROADMAP FOR FUTURE DEVELOPMENT

Phase	Focus Area	Expected Benefit
Phase 1	Multi-channel (RGB) Image Support	Enhanced visual processing and broader applicability
Phase 2	Video Stream Integration with Real-time Filtering	Live use in surveillance and automotive systems
Phase 3	Integration with Deep Learning Models	Intelligent post-processing and classification
Phase 4	Multi-GPU and Cloud Deployment	High-performance parallel processing at scale

3) Technology Evolution Roadmap:

B. Research Implications

1) Potential Industry Applications: The results of this project have direct implications across multiple industries:

- **Healthcare Imaging:** Real-time filtering and edge detection can assist in diagnostic systems like MRI or CT scan enhancement.
- **Automotive Sector:** Sobel edge detection and sharpening can enhance visual data from self-driving vehicle sensors.
- **Security and Surveillance:** Accelerated GPU-based filtering is ideal for processing video streams in intrusion detection and anomaly monitoring.
- **Augmented Reality:** Real-time processed imagery is vital for AR applications, especially on wearable devices.
- **Manufacturing and Robotics:** Enhanced visual pipelines can aid in fault detection and object recognition for automation.

Future research can focus on integrating traditional GPU-accelerated filtering techniques with neural network models (e.g., U-Net, YOLO) to build intelligent, hybrid computer vision systems that are both fast and context-aware.

2) Environmental Impact and Energy Efficiency: Energy efficiency is a growing concern in high-performance computing, especially as large-scale deployments become the norm. Although GPUs consume more power per unit than CPUs, they are significantly more efficient in terms of **work done per watt** when parallel workloads are involved.

As the table illustrates, while GPUs demand more power at a given moment, they complete tasks in significantly less time, resulting in a **net decrease in energy consumption per task**. This makes CUDA-based solutions more sustainable for large-scale or real-time image processing pipelines.

TABLE V
ENERGY EFFICIENCY COMPARISON: CPU VS GPU

Metric	CPU (OpenCV)	GPU (CUDA)
Power Draw	15–45 W (per core)	75–250 W (per GPU)
Execution Time	Higher for parallel tasks	Significantly lower for same tasks
Energy per Task	High (longer time × lower power)	Lower (shorter time × higher power)
Thermal Output	Minimal cooling needed	Requires active cooling systems
Eco-Friendliness (in batch)	Less efficient for large-scale processing	More eco-efficient due to faster throughput

In future implementations, additional energy-saving techniques like dynamic voltage scaling, adaptive workloads, and cloud-based GPU scheduling can be explored to further enhance environmental sustainability.

VI. CONCLUSION

A. Recap of Key Findings

This project successfully explored the use of CUDA, NVIDIA's parallel computing platform, to accelerate common image processing algorithms including Gaussian Blur, Sobel Edge Detection, and Image Sharpening. Through side-by-side implementations in both CPU (OpenCV) and GPU (CUDA), we demonstrated significant performance improvements, with speedup factors ranging from approximately 6x to 12x.

The quantitative results established that GPU-based processing is not only faster but also scalable and suitable for real-time applications. Qualitatively, the output images from CUDA kernels were visually comparable to their CPU-based equivalents, ensuring no trade-off in accuracy or visual fidelity.

B. Overall Project Significance

The significance of this project lies in its practical demonstration of how parallel computing can be leveraged to optimize computationally intensive tasks in image processing. With applications spanning across industries such as healthcare, automotive, security, and augmented reality, GPU acceleration provides a viable and sustainable alternative to traditional serial processing.

Furthermore, the modular design of the implementation lays the groundwork for future research in integrating CUDA with more advanced techniques such as machine learning, multi-GPU systems, and real-time video stream processing. This project not only validates CUDA's capabilities for performance enhancement but also encourages deeper exploration into scalable, efficient, and eco-friendly computing paradigms.

REFERENCES

[1] Y. Zhang and Y. Zhang, "Real-time edge detection using cuda," *Journal of Real-Time Image Processing*, vol. 8, no. 1, pp. 67–77, 2012.

[2] C. Munteanu and M. Radoi, "Performance comparison of cpu and gpu in image processing algorithms," *Scientific Bulletin of the Politehnica University of Timișoara*, vol. 57, no. 71, pp. 23–28, 2012.

[3] N. K. Govindaraju and J. D. Owens, "Gpu algorithms for accelerating image convolution," in *GPU Gems 2*. Addison-Wesley, 2005, pp. 879–893.

[4] R. Shams, R. A. Kennedy, R. I. Hartley, and et al., "A survey of medical image registration on multicore and gpu," *Journal of Parallel and Distributed Computing*, vol. 68, no. 10, pp. 1307–1321, 2008.

[5] X. Luo, H. Li, and Y. Wang, "Gpu-accelerated real-time image sharpening for high-resolution video," *IEEE Transactions on Multimedia*, vol. 24, pp. 2205–2216, 2022.

[6] J. Xu, Q. Liu, and W. Zhang, "Acceleration of bilateral filtering using cuda for real-time applications," *IEEE Access*, vol. 8, pp. 33 123–33 132, 2020.